

Backend API Integration Guide

Main integration files (edit these first):

- `src/api/endpoints.js` — all endpoint paths
- `src/api/index.js` — all API methods (auth, sop, pipeline, cameras, reports)
- `src/config/api.js` — base URL & port (`http://<host>:8000`)

This document lists every API the Compliance Monitoring frontend expects.

Share this with the backend team — UI pages import via `src/api/` or thin wrappers in `src/services/`.

Base URLs

Config | Value | File

REST API | `http://<host>:8000` | `src/config/api.js`

WebSocket | `ws://<host>:8000` | `src/config/api.js`

Change port/host in `src/config/api.js` only — UI components do not hardcode URLs.

Frontend service map

File | Role

`src/api/endpoints.js` | Endpoint paths — backend team edits here

`src/api/index.js` | Main API module — all methods

`src/api/client.js` | Shared fetch + error handling

`src/config/api.js` | Base URL (`http://host:8000`)

`src/services/*.js` | Thin wrappers (backward compatible)

```
import api from '../api';

await api.sop.getConfigs();
await api.sop.saveConfig(2, ['check person wearing gloves.']);
await api.pipeline.start();
```

1. SOP Workflows (priority — main feature)

Reference backend: `dummy-backend.py` — endpoints exist.

GET `/api/sop-config`

Load all per-camera SOP sequences.

Response:

```
{
  "streams": [
    {
      "stream_id": 1,
      "sop_steps": [
        "check person wearing gloves.",
        "check person wearing vest."
      ],
      "updated_at": "2026-07-21T12:00:00Z"
    }
  ]
}
```

PATCH `/api/sop-config`

Create or update one camera's SOP sequence.

Request body:

```
{
  "stream_id": 2,
  "sop_steps": [
    "check person wearing gloves.",
    "check person wearing vest."
  ]
}
```

Response:

```
{
  "status": "success",
  "config": {
    "stream_id": 2,
    "sop_steps": [...],
    "updated_at": "..."
  }
}
```

Notes:

- sop_steps must not be empty (400 if empty).
- Order of array = enforcement order in pipeline.
- Each step is a plain string (period at end is optional; UI adds . on save).

DELETE /api/sop-config/{stream_id}

Remove SOP config for a stream.

Example: DELETE /api/sop-config/2

Response:

```
{ "status": "success", "stream_id": 2 }
```

404 if no config exists for that stream.

2. SOP Reports

GET /api/sop-events

List SOP compliance events for the reports table.

Query parameters (all optional):

Param	Type	Description
stream_id	int	Filter by camera
object_id	int	Filter by tracked person
only_alerts	bool	If true: skipped, out_of_order, overdue only
limit	int	Default 100
offset	int	Pagination offset

Response: array of:

```
{
  "id": 1,
  "timestamp": "Jul 21, 2026 - 09:12:08 AM",
  "stream_id": 1,
  "object_id": 3,
  "event_type": "completed",
  "step_index": 1,
  "step_text": "check person wearing gloves.",
  "raw_vlm_response": "Worker is wearing gloves.",
  "image_path": "stored_sop_images/sop_1_person3....jpg"
}
```

event_type values: completed | skipped | out_of_order | overdue

GET /api/sop-events/person-ids

Distinct person IDs for the Person filter dropdown.

Query: ?stream_id=1 (optional)

Response:

```
{ "object_ids": [3, 5, 7, 12] }
```

Static images

```
GET /stored_sop_images/{filename}
```

Frontend builds URL from image_path in each event (see src/utils/sopEventUtils.js !)

WebSocket — live SOP events

```
WS /ws/violations
```

Frontend listens for messages where kind === "sop_event":

```
{
  "kind": "sop_event",
  "id": 1,
  "timestamp": "...",
  "stream_id": 1,
  "object_id": 3,
  "event_type": "skipped",
  "step_index": 2,
  "step_text": "...",
  "raw_vlm_response": "...",
  "image_path": "..."
}
```

Hook: src/hooks/useSopEvents.js

3. Pipeline & live dashboard

Method | Endpoint | Response (expected)

POST | /api/start-pipeline | { status, message, external_ip? }

POST | /api/stop-pipeline | { status, message }

GET | /api/pipeline-status | { deepstream_running: bool, go2rtc_running: bool }

GET | /api/stream-config | { streamUrl: "http://IP:1984/stream.html?src=..." }

Service: src/services/pipelineService.js

UI: Dashboard ! Start/Stop Stream, camera iframe tiles

4. PPE reports (legacy)

GET /api/reports

Query: limit, offset

Response: array of:

```
{
  "id": 1001,
  "timestamp": "Jul 07, 2026 - 11:25:28 AM",
  "label": "no-mask",
  "stream_id": 4,
  "confidence": 0.548,
  "image_path": "stored_images/mock_frame_4a.jpg"
}
```

GET /api/reports/download-pdf

Query:

Param | Format | Example

start_date | MM-DD-YYYY | 07-07-2026
end_date | MM-DD-YYYY | 07-09-2026
start_time | optional HH:MM | 08:00
end_time | optional HH:MM | 18:00

Response: application/pdf blob

Static images

```
GET /stored_images/{filename}
```

Service: src/services/reportService.js

UI: PPE Reports page

5. Cameras — not in dummy-backend.py yet

Frontend calls these; falls back to hardcoded Camera 1–5 if API fails.

Method | Endpoint | Body / response

GET | /api/cameras | [{ id, name, rtsp_url, status }]

POST | /api/cameras | Body: { name, rtsp_url } !' created camera object

DELETE | /api/cameras/{id} | —

Service: src/services/cameraService.js

UI: Add Stream page

Suggested status values: online | offline | violation_active

6. Auth — not in dummy-backend.py yet

Method | Endpoint | Body

POST | /api/auth/login | { username, password }

POST | /api/auth/logout | —

GET | /api/auth/me | Returns current user session

Service: src/services/authService.js

Note: In dev, any non-empty login works if auth API is unavailable.

7. Optional violations API

Frontend tries these first, then falls back to /api/reports:

Method | Endpoint

GET | /api/violations?camera_id=&limit=

GET | /api/violations/{id}

POST | /api/violations/{id}/acknowledge

Service: src/services/violationService.js

8. Pipeline-only (backend internal — UI does not call)

Endpoint | Called by

POST /api/ingest-sop-event | DeepStream SOP pipeline

POST /api/ingest-violation | PPE detection pipeline

GET /api/internal/pending-captures | Pipeline workers

POST /api/internal/frame-capture-result | Pipeline workers

Implementation checklist

Already in dummy-backend.py

- [x] GET /api/sop-config
- [x] PATCH /api/sop-config
- [x] DELETE /api/sop-config/{stream_id}
- [x] GET /api/sop-events
- [x] GET /api/sop-events/person-ids
- [x] POST /api/ingest-sop-event
- [x] WS /ws/violations (push kind: "sop_event")
- [x] /stored_sop_images static mount
- [x] Pipeline: start / stop / status / stream-config
- [x] PPE: /api/reports, /api/reports/download-pdf
- [x] /stored_images static mount

Backend team should add

- [] GET/POST/DELETE /api/cameras
- [] POST /api/auth/login, /api/auth/logout, GET /api/auth/me
- [] (Optional) /api/violations REST API

CORS & errors

- Enable CORS for the frontend origin (dev: http://localhost:3000).
- Error responses: JSON { "detail": "message" } or FastAPI validation array.
- Frontend client: src/services/apiClient.js parses detail for user-facing errors.

Contact points for endpoint changes

If the backend changes a path or payload, update src/api/endpoints.js and src/api/index.js — no UI component changes required.